Lista 4: ordenação e busca

Encontro 6, de 23/09

Estruturas de dados e análise de algoritmos

UC 0006963 · Unicuritiba · 2026/2

Declaração de uso de IA: usei o Claude (Anthropic) para ajudar na questão 9 com o código. Linguagem declarada: Python.

Como entregar

- Entrega até a véspera do encontro de 07/10, pelo AVA.
- Esta lista compõe os 10% da nota da A3 destinados às listas de exercícios. As questões também são o modelo das que caem na A1.
- Os rastreios devem vir desenhados, com as divisões e as junções visíveis.
- Nas questões de implementação, use a linguagem que você declarou, e instrumente com contador de comparações.
- Uso de IA conforme o contrato: permitido, declarado, e com capacidade de explicar linha a linha.

Questões

Questão 1. Rastreie o insertion sort no vetor abaixo, mostrando o estado após cada iteração do laço externo e indicando quantos deslocamentos ocorreram em cada uma.

```
[ 7  3  9  2  8  1 ]
```

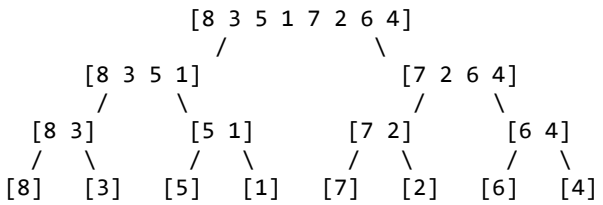
```
inicial           [ 7 3 9 2 8 1 ]
i=1 (chave 3)     [ 3 7 | 9 2 8 1 ]   1 deslocamento
i=2 (chave 9)     [ 3 7 9 | 2 8 1 ]   0 deslocamentos
i=3 (chave 2)     [ 2 3 7 9 | 8 1 ]   3 deslocamentos
i=4 (chave 8)     [ 2 3 7 8 9 | 1 ]   1 deslocamento
i=5 (chave 1)     [ 1 2 3 7 8 9 ]   5 deslocamentos
Total: 10 deslocamentos
```

A barra | separa a parte já ordenada do restante.

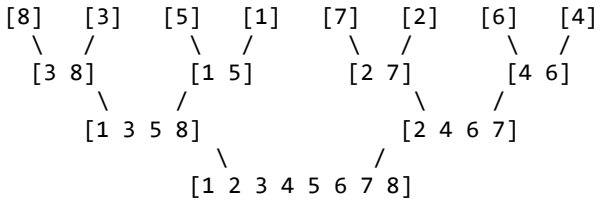
Questão 2. Rastreie o merge sort no vetor abaixo. Desenhe todas as divisões até chegar a vetores de um elemento e depois todas as intercalações. Indique quantos níveis existem e por quê.

```
[ 8  3  5  1  7  2  6  4 ]
```

Divisões:



Intercalações:



São **4 níveis** (8 → 4 → 2 → 1 elementos), que é $\log^2 8 + 1$. Cada divisão corta o vetor ao meio, então bastam 3 divisões para chegar a vetores de um elemento; a altura da árvore cresce com $\log^2 n$.

Questão 3. Intercale as duas listas ordenadas abaixo, passo a passo, indicando qual elemento sai em cada comparação. Ao final, diga quantas comparações foram feitas e qual é o número máximo de comparações possível ao intercalar duas listas de tamanhos m e n.

```
lista A: [ 2  5  8  11 ]
lista B: [ 1  3  9  12 ]
```

```
1) 2 x 1  → sai 1 (de B)   saída: [ 1 ]
2) 2 x 3  → sai 2 (de A)   saída: [ 1 2 ]
3) 5 x 3  → sai 3 (de B)   saída: [ 1 2 3 ]
4) 5 x 9  → sai 5 (de A)   saída: [ 1 2 3 5 ]
5) 8 x 9  → sai 8 (de A)   saída: [ 1 2 3 5 8 ]
6) 11 x 9  → sai 9 (de B)   saída: [ 1 2 3 5 8 9 ]
7) 11 x 12  → sai 11 (de A)   saída: [ 1 2 3 5 8 9 11 ]
Sobra 12 (de B), copiado sem comparar.
Resultado: [ 1 2 3 5 8 9 11 12 ]
```

Foram **7 comparações**. No máximo, ao intercalar listas de tamanhos m e n, são **m + n − 1**: cada comparação faz sair um elemento, e o último sempre sai sem precisar ser comparado. Aqui m + n − 1 = 7, então esse foi o pior caso.

Questão 4. No vetor abaixo, liste as posições visitadas pela busca binária ao procurar o 13, e depois ao procurar o 4. Indique quantas comparações cada busca fez.

```
[ 1  3  5  7  9  11  13  15 ]
```

```
posição:  0  1  2  3  4  5   6   7
valor:    1  3  5  7  9  11  13  15
```

Buscando 13: 3 (valor 7) → 5 (valor 11) → 6 (valor 13). 3 comparações, achou na posição 6.

Buscando 4: 3 (valor 7) → 1 (valor 3) → 2 (valor 5). 3 comparações, não encontrado (devolve -1).

Posições contadas a partir de 0, uma comparação por posição visitada.

Questão 5. Aplique a busca binária ao vetor desordenado abaixo, procurando o 9. Mostre o caminho percorrido e o resultado devolvido. Em três linhas, explique por que essa falha é mais perigosa que uma exceção.

```
[ 7  2  9  1  5  8  3  6 ]
```

```
posição:  0  1  2  3  4  5  6  7
valor:    7  2  9  1  5  8  3  6

low=0 high=7 mid=3 → valor 1 < 9, descarta a esquerda
low=4 high=7 mid=5 → valor 8 < 9, descarta a esquerda
low=6 high=7 mid=6 → valor 3 < 9, descarta a esquerda
low=7 high=7 mid=7 → valor 6 < 9, descarta a esquerda
low=8 > high=7 → para e devolve -1
```

Caminho: posições 3, 5, 6, 7. Resultado: **-1** (não encontrado), mas o 9 está na posição 2.

A busca binária só funciona se o vetor estiver ordenado; sem isso ela descarta a metade errada e não percebe.

O resultado -1 parece uma resposta normal, então o programa segue em frente acreditando que o 9 não existe.

Uma exceção para o programa e avisa que algo deu errado; essa falha é silenciosa e contamina o que vier depois.

Questão 6. Para cada recorrência, diga a classe de complexidade e cite um algoritmo visto em aula que a produz.

```
T(n) = T(n/2) + c
```

O(log n). Busca binária: a cada passo descarta metade e faz uma comparação (c).

```
T(n) = T(n-1) + c
```

O(n). Busca sequencial (ou percurso recursivo de uma lista): um passo de custo c e uma chamada para n − 1 elementos.

```
T(n) = 2 T(n/2) + n
```

O(n log n). Merge sort: duas chamadas de tamanho n/2 mais a intercalação, que custa n.

Questão 7. Um sistema tem um vetor de 1 milhão de registros não ordenados. Considere que ordenar custa cerca de n log n comparações e cada busca binária custa log n.

Com $n = 10^6$, $\log^2 n \approx 20$. Ordenar custa cerca de $10^6 \times 20 = 2 \times 10^7$ comparações. Sem ordenar, cada consulta é uma busca linear: 10^6 no pior caso (5×10^5 em média).

- a) Para 5 consultas, compensa ordenar antes? Mostre a conta.

a) Não compensa. Ordenando: $2 \times 10^7 + 5 \times 20 \approx 2 \times 10^7$. Sem ordenar: 5×10^6 no pior caso ($2{,}5 \times 10^6$ em média). Só a ordenação já custa mais que as cinco buscas lineares.

- b) E para 100 mil consultas? Mostre a conta.

b) Compensa muito. Ordenando: $2 \times 10^7 + 10^5 \times 20 = 2{,}2 \times 10^7$. Sem ordenar: $10^5 \times 10^6 = 10^{11}$ no pior caso (5×10^{10} em média). É mais de 2.000 vezes menos trabalho.

- c) Estime, aproximadamente, a partir de quantas consultas ordenar passa a compensar.

c) Precisa de $k \times 10^6 > 2 \times 10^7 + 20k$, ou seja, $k > 20$ no pior caso ($k > 40$ se usar o custo médio de 5×10^5 por busca linear). Então, a partir de **20 a 40 consultas** (em ordem de grandeza, log n consultas) ordenar já vale a pena.

Questão 8. Para cada cenário, escolha entre insertion sort, merge sort e quick sort, e justifique em uma linha.

- a) Ordenar 20 nomes digitados num formulário.

a) Insertion sort. Com 20 elementos o custo quadrático é irrelevante e o código é simples, sem overhead.

- b) Ordenar 50 milhões de registros de log que não cabem na memória.

b) Merge sort. Ordena blocos que cabem na memória e intercala sequencialmente, além de garantir n log n no pior caso.

- c) Ordenar uma lista quase ordenada, com dois elementos fora de lugar.

c) Insertion sort. Em lista quase ordenada ele faz poucos deslocamentos e fica próximo de O(n).

- d) Ordenar registros em que a ordem original entre elementos iguais precisa ser preservada.

d) Merge sort. É estável (no empate sai primeiro o elemento da esquerda); o quick sort comum não garante isso.

Questão 9. Implemente insertion sort e merge sort, ambos instrumentados com contador de comparações. Gere a tabela de comparações para n igual a 100, 1.000, 10.000 e 100.000, com entrada aleatória. Em até dez linhas, responda: a partir de qual n o merge sort compensa, e por que ele perde para n pequeno?

Implementação em Python (cada comparação entre elementos incrementa o contador):

```
import random

def insertion_sort(v):
    a = v[:]
    comps = 0
    for i in range(1, len(a)):
        chave = a[i]
        j = i - 1
        while j >= 0:
            comps += 1                  # compara a[j] com a chave
            if a[j] > chave:
                a[j + 1] = a[j]
                j -= 1
            else:
                break
        a[j + 1] = chave
    return a, comps

def merge_sort(v):
    if len(v) <= 1:
        return v[:], 0
    meio = len(v) // 2
    esq, ce = merge_sort(v[:meio])
    dir, cd = merge_sort(v[meio:])
    res, comps = [], ce + cd
    i = j = 0
    while i < len(esq) and j < len(dir):
        comps += 1                      # uma comparação por elemento que sai
        if esq[i] <= dir[j]:
            res.append(esq[i]); i += 1
        else:
            res.append(dir[j]); j += 1
    res += esq[i:] + dir[j:]
    return res, comps

if __name__ == "__main__":
    random.seed(42)
    print(f"{'n':>8} {'insertion':>15} {'merge':>12}")
    for n in (100, 1000, 10000, 100000):
        v = [random.randint(0, 10**9) for _ in range(n)]
        _, cm = merge_sort(v)
        _, ci = insertion_sort(v)
        print(n, ci, cm)
```

Comparações, entrada aleatória (seed 42):

n	Insertion sort	Merge sort
100	2.444	557
1.000	245.354	8.704

10.000	25.013.579	120.397
100.000	2.495.315.512	1.536.521

O valor de n = 100.000 do insertion sort foi contado com o mesmo algoritmo portado para C sobre a mesma entrada, porque em Python leva muitos minutos.

Em número de comparações o merge sort já compensa em n = 100 (557 contra 2.444), e a distância só cresce: em 100.000 são cerca de 1,5 milhão contra quase 2,5 bilhões. Testando n menores, o cruzamento fica em torno de n = 5; a partir daí o merge faz menos comparações em média. Ainda assim ele perde para n pequeno na prática porque cada passo custa mais: tem recursão, cópia de sublistas e vetor auxiliar de tamanho n, enquanto o insertion sort trabalha no próprio vetor, num laço simples. Por isso as bibliotecas costumam usar insertion sort em pedaços pequenos (algo entre 10 e 50 elementos).

Questão 10. Discursiva. Explique, em até dez linhas e sem usar a prova formal, por que nenhum algoritmo de ordenação baseado em comparações pode ser melhor que n log n no pior caso. Use o argumento do número de ordens possíveis e da informação obtida por comparação.

Se tenho n elementos, eles podem estar embaralhados de n! jeitos diferentes (n × (n−1) × ... × 1), e o algoritmo precisa descobrir qual deles é o da minha entrada. Ele só consegue isso comparando elementos, e cada comparação tem apenas duas respostas possíveis: sim ou não. Então, no melhor dos casos, cada comparação elimina metade das possibilidades, como num jogo de adivinhação em que só se pode fazer perguntas de sim ou não. Para chegar a uma única ordem entre n! possíveis, é preciso fazer pelo menos tantas perguntas quantas são as vezes que dá para dividir n! ao meio. Esse número cresce na mesma proporção que n log n. Por isso, no pior caso, nenhum algoritmo que ordena por comparações consegue ficar abaixo de n log n, e o merge sort já chega nesse limite.

Antes de entregar

Confira se os rastreios das questões 1 a 3 vieram desenhados, e não apenas descritos.

Confira se a tabela da questão 9 traz contagem de comparações, e não tempo de relógio.

Se você usou IA em alguma questão, declare no topo do arquivo o que usou e para quê.